

10 — CAP + AI Core Integration + UI5 Frontend (Deep Dive)

Module purpose: the other half of sheet 09 — how the CAP service layer actually calls SAP AI Core, and how a SAPUI5 app calls CAP. Closes the loop: **UI5 → CAP OData/action → AI Core → HANA → back to UI5.**

End-to-End Call Chain

UI5 Controller (JS)

```
-- calls OData action --> CAP Service (runtime.js)
  -- calls --> AI Core (Orchestration Service, sheets 07-08 concepts)
  -- reads/writes --> HANA (CQL, sheet 09)
<-- returns RagResponse --
<-- updates JSONModel / OData model --
```

CAP Service Implementation Calling AI Core — srv/runtime.js

```
module.exports = cds.service.impl(async function () {
  const capllmplugin = await cds.connect.to("cap-llm-plugin"); // destination-backed AI
    ↪ Core connector

  this.on("getChatRagResponse", async (req) => {
    const { user_query, conversationId } = req.data;

    // Step 1 – embed the query (Node.js equivalent of sheet 04's embedding_setup.py)
    const embeddingConfig = cds.env.requires["gen-ai-hub"]["text-embedding-ada-002"];
    const embedding = await capllmplugin.getEmbeddingWithConfig(embeddingConfig,
      ↪ user_query);

    // Step 2 – similarity search in HANA (equivalent of sheet 04's similarity_search.py)
    const chunks = await capllmplugin.similaritySearch(
      "DocumentChunk", "embedding", "text_chunk", embedding, "COSINE_SIMILARITY", 5);

    // Step 3 – call the LLM with retrieved context (equivalent of sheet 05's
      ↪ rag_pipeline.py)
    const chatResponse = await capllmplugin.getHarmonizedChatCompletion({
      clientConfig: chatCompletionConfig, getContent: true,
    });

    // Step 4 – persist via CQL (sheet 09)
    await INSERT.into(Message).entries({ cID: conversationId, content: chatResponse,
      ↪ role: "assistant" });

    return { message: chatResponse, sources: chunks };
  });
});
```

Working explanation: `cds.connect.to("cap-llm-plugin")` is CAP's dependency-injection style of getting a configured client — same JOB as sheet 02's `get_proxy_client("gen-ai-hub")`, just a Node.js-idiomatic API instead of a Python function call. The four steps here are a **line-by-line match** to the Python RAG pipeline: embed → similarity search → LLM call with context → persist — proof that the underlying concepts (embedding, cosine similarity search, grounded LLM calls) are identical regardless of runtime; only the syntax and library names change.

AI Core Destination Config — package.json

```
"cds": { "requires": { "gen-ai-hub": {  
  "kind": "GenAIHubDestination",  
  "destination": "aicore-destination"  
}}}
```

Working explanation: this is the CAP-world equivalent of sheet 01/02's .env + AICORE_CLIENT_ID/SECRET pair — instead of raw environment variables read by python-dotenv, credentials live in a BTP **Destination** object, resolved by the platform at runtime (typically pointing to an aicore-destination configured with OAuth2ClientCredentials against https://api.ai.<region>.<provider>.ml.hana.ondemand.com). Same underlying OAuth2 client-credentials flow as get_proxy_client, different place the secret is stored.

UI5 App Wiring — app/chat-ui/webapp/

```
webapp/  
├─ manifest.json          <- declares dataSources: mainService -> /odata/v4/chat/  
├─ Component.js  
├─ controller/App.controller.js  
└─ view/App.view.xml
```

```
// manifest.json (excerpt) — declares WHERE the OData service lives  
"dataSources": { "mainService": { "uri": "/odata/v4/chat/", "type": "OData", "settings":  
  ↪ {"odataVersion": "4.0"} } }
```

```
// App.controller.js — calling the CAP action from UI5  
onSend: function () {  
  const oModel = this.getView().getModel(); // the OData V4 model bound  
  ↪ via manifest.json  
  const oAction = oModel.bindContext("/getChatRagResponse(...)", null, { $$updateGroupId:  
    ↪ "submitLater" });  
  oAction.setParameter("user_query", this._oInputField.getValue());  
  oAction.execute().then((oResult) => {  
    this._appendMessageToChat(oResult.getObject()); // update the bound chat view  
  });  
},
```

Working explanation: bindContext(...).execute() is UI5's OData V4 mechanism for invoking a custom **action** (as opposed to just reading an entity set with bindList) — the UI5 equivalent of the Python templates' agent.ask(question) call. The framework model handles serialization, the HTTP round trip, and updating any bound UI controls automatically once .execute() resolves — no manual fetch()/XMLHttpRequest and JSON parsing required, similar to how LangChain's chain.invoke() hides the underlying HTTP call to SAP AI Core in the Python templates.

Deployment — mta.yaml (Multi-Target Application)

```
modules:  
- name: my-app-srv          # CAP Node.js service  
  type: nodejs  
- name: my-app-approuter    # serves the UI5 app, routes to srv, enforces auth  
  type: approuter.nodejs  
- name: my-app-db-deployer  # deploys db/schema.cds (sheet 09) to the HDI container  
  type: hdb
```

```
resources:
- name: my-app-xsuaa           # auth – from xs-security.json
- name: my-app-destination    # AI Core / other external system credentials
- name: my-app-hdi            # the HANA HDI container itself
```

Working explanation: the **approuter** is the single production entry point — it serves the UI5 static files AND reverse-proxies /odata/... calls to the CAP srv module, attaching the authenticated user's XSUAA JWT to each forwarded request. The browser (UI5 app) never talks to srv directly in production; it only ever talks to the approuter, which is the CAP-world counterpart of never letting a frontend hold AI Core credentials directly. Local development uses a [hybrid] MTA profile with mocked auth (fast iteration without deploying); [production] binds the real xsuaa resource — same “local convenience, production security” split as .env vs. VCAP_SERVICES in sheet 01.

Key Points

- The **approuter** is the single entry point in production: serves UI5 static files, proxies OData calls to CAP, and enforces XSUAA auth — the browser never talks to srv directly.
- AI Core credentials live in a **BTP Destination**, resolved by the platform, not a .env file read by application code — same OAuth2 client-credentials concept as sheet 01/02, different storage.
- The UI5 side only ever calls **your own CAP service** — never SAP AI Core directly — keeping credentials and deployment IDs fully server-side.
- bindContext(...).execute() invokes an OData V4 **action** — the UI5 equivalent of Python's agent.ask(question) / bot.chat(message) call pattern.
- [hybrid] vs [production] MTA profiles mirror the .env-vs-bound-service pattern from sheet 01 — fast local iteration without sacrificing production security posture.
- Every CAP/Node.js piece here has a **direct Python counterpart** in sheets 01-08 — same concepts (auth, embedding, similarity search, grounded LLM calls, persistence), different runtime/syntax.

Interview Q&A

Q: Why route the UI5 app's AI calls through CAP instead of calling AI Core directly from the browser? A: Keeps AI Core credentials/deployment IDs server-side only, lets you apply masking/filtering/auth/rate-limiting centrally in one place, and avoids exposing internal service topology (deployment URLs, resource groups) to the client.

Q: What is the approuter's specific role in this architecture, and why is it a separate MTA module? A: Single entry point that serves static UI5 files, reverse-proxies OData calls to the CAP service, and enforces XSUAA authentication before any request reaches srv — separating it as its own module lets it scale/deploy independently and keeps auth enforcement in one auditable place.

Q: How does this CAP+UI5 project structure map back to the plain-Python templates in sheets 01-08? A: db/schema.cds (sheet 09) ≈ sheet 04's vector table DDL; runtime.js's AI Core calls ≈ sheets 02/05's llm_setup.py/rag_pipeline.py; CQL ≈ sheet 01/03's hdbcli SQL; the UI5 controller ≈ sheet 06's CLI runner/Streamlit app as the presentation layer — same architecture, different language/runtime.

Q: What would break if you skipped the approuter and pointed the UI5 app straight at the CAP srv module's URL? A: You'd lose centralized auth enforcement and static file serving in one place, and the frontend would need to handle CORS and JWT attachment itself — the approuter exists specifically to remove that burden from both the frontend and the backend service.